

Отчёт по лабораторной работе №2

первоначальная настройка git

Юсупова Алина Руслановна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
3	Вывод	20
4	Контрольные вопросы	21

Список иллюстраций

2.1	Загрузка пакетов	7
2.2	Загрузка пакетов	8
2.3	Параметры репозитория	9
2.4	Создание ssh ключа	10
2.5	Создание GPG ключа	11
2.6	GPG ключ	12
2.7	Добавление GPG ключа	13
2.8	Параметры репозитория	14
2.9	Связь репозитория с аккаунтом	15
2.10	Успешный вход на git	16
2.11	Загрузка шаблона	17
2.12	Первый коммит	18
2.13	Первый коммит	19

Список таблиц

1 Цель работы

Целью данной работы является изучение идеологии и применения средств контроля версий и освоение умений работать с git.

2 Выполнение лабораторной работы

Устанавливаем git и gh.

```
ayusupova@ayusupova:~$ sudo -i
(sudo) пароль для ayusupova:
root@ayusupova:~# df install git
Обновление и загрузка репозитория:
Fedora 43 - x86_64 - Updates 100% | 30.9 KiB/s | 17.8 KiB | 00m01s
Репозитории загружены.
Пакет "git-2.53.0-1.fc43.x86_64" уже установлен.

Нечего делать.
root@ayusupova:~#
```

Рисунок 2.1: Загрузка пакетов

```
root@atysupova:~# dnf install gh
Обновление и загрузка репозитория:
Репозитории загружены.
Пакет                                Арх.      Версия                                Репозиторий      Размер
Установка:
gh                                  x86_64    2.87.0-2.fc43                       updates           37.6 MiB

Сводка транзакции:
Установка:      1 пакета

Общий размер входящих пакетов составляет 11 MiB. Необходимо загрузить 11 MiB.
После этой операции будут использоваться дополнительные 38 MiB (установка 38 MiB, удаление 0 B).
Is this ok [y/N]: y
[1/1] gh-0:2.87.0-2.fc43.x86_64                                100% | 5.2 MiB/s | 11.0 MiB | 00m02s

-----
[1/1] Всего                                                    100% | 3.1 MiB/s | 11.0 MiB | 00m04s
Выполнение транзакции
[1/3] Проверить файлы пакета                                    100% | 14.0 B/s | 1.0 B | 00m00s
[2/3] Подготовить транзакцию                                    100% | 4.0 B/s | 1.0 B | 00m00s
[3/3] Установка gh-0:2.87.0-2.fc43.x86_64                    100% [=====] | 38.8 MiB/s | 37.7 MiB | 00m00s
```

Рисунок 2.2: Загрузка пакетов

Зададим имя и email владельца репозитория, кодировку и прочие параметры.


```
root@aryusupova:~# git config --global user.name 'alyusupova'
root@aryusupova:~# git config --global user.email '1032233510@pfur.ru'
root@aryusupova:~# git config --global core.quotepath false
root@aryusupova:~# git config --global init.defaultBranch master
root@aryusupova:~# git config --global core.autocrlf input
root@aryusupova:~# git config --global core.safecrlf warn
root@aryusupova:~#
```

Рисунок 2.3: Параметры репозитория

Создаем SSH ключи

```
[sudo] пароль для aryusupova:

root@aryusupova:~# ls ~/.ssh/

id_rsa  id_rsa.pub

root@aryusupova:~# cat ~/.ssh/id_rsa.pub
```

Рисунок 2.4: Создание ssh ключа

Создаем GPG ключ

```

root@aryusupova:~# gpg --full-generate-key
gpg (GnuPG) 2.4.8; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

gpg: создан каталог '/root/.gnupg'
Выберите тип ключа:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (только для подписи)
 (14) Existing key from card
Ваш выбор? 1
длина ключей RSA может быть от 1024 до 4096.
Какой размер ключа Вам необходим? (3072) 4096
Запрошенный размер ключа - 4096 бит
Выберите срок действия ключа.
    0 = не ограничен
    <n> = срок действия ключа - n дней
    <n>w = срок действия ключа - n недель
    <n>m = срок действия ключа - n месяцев
    <n>y = срок действия ключа - n лет
Срок действия ключа? (0) 0
Срок действия ключа не ограничен
Все верно? (y/N) y

GnuPG должен составить идентификатор пользователя для идентификации ключа.

Ваше полное имя: Alina
Адрес электронной почты: 1032253510@pfur.ru
Примечание:
Вы выбрали следующий идентификатор пользователя:
    "Alina <1032253510@pfur.ru>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход?

```

Рисунок 2.5: Создание GPG ключа

```

"alyusupova <1032253510@pfur.ru>"

Сменить (N)Имя, (C)Примечание, (E)Адрес; (O)Принять/(Q)Выход? O
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
Необходимо получить много случайных чисел. Желательно, чтобы Вы
в процессе генерации выполняли какие-то другие действия (печать
на клавиатуре, движения мыши, обращения к дискам); это даст генератору
случайных чисел больше возможностей получить достаточное количество энтропии.
gpg: создан каталог '/home/alyusupova/.gnupg/openpgp-revocs.d'
gpg: сертификат отзыва записан в '/home/alyusupova/.gnupg/openpgp-revocs.d/F42E246A3
75A7CC2C516299E73EEF2CF4DBD38BA.rev'.
открытый и секретный ключи созданы и подписаны.

pub  rsa4096 2026-02-27 [SC]
      F42E246A375A7CC2C516299E73EEF2CF4DBD38BA
uid          alyusupova <1032253510@pfur.ru>
sub  rsa4096 2026-02-27 [E]

```

Рисунок 2.6: GPG ключ

Добавляем GPG ключ в аккаунт

Add new GPG key

Title

Key

```
-----BEGIN PGP PUBLIC KEY BLOCK-----  
  
mQINBGmh2YYBEAC0BaVCluoiP0FgzBwLRqs2gRn5YHDMxTayZTLT  
z/nHgRt16mtu  
T9TCaIvdEbjNHp8BLaLCESdMIOMztdKVjjeGZn+dAROnrA9N7wjA2G  
8d9loof+1o  
6KXrup9APanRXAIhSSr7FDqT3vIET1QmKz1ddDpGrWfL0UbIomHBk  
LFkCA65VIDI  
qp26Y2khrj7dcicngeOQaxlmmX5oh8t8bURwqjPuMMnCgB9uaAkP5
```

Add GPG key

[Terms](#) [Privacy](#) [Security](#) [Status](#) [Community](#) [Docs](#) [Contact](#)

Рисунок 2.7: Добавление GPG ключа

Настройка автоматических подписей коммитов git

```
arvusudova@arvusudova:~$ git config --global user.signingkey 42E246A375A7C025162997BEE2  
C40B0308A  
arvusudova@arvusudova:~$ git config --global commit.gpgsign true  
arvusudova@arvusudova:~$ git config --global gpg.program $(which gpg)  
arvusudova@arvusudova:~$
```

Рисунок 2.8: Параметры репозитория

Настройка gh

```
libEGL warning: egl: failed to create dri2 screen

aryusupova@aryusupova:~$ git config --global user.signingkey F42E246A375A7CC2C516299E73EEF2CF4DBD38BA
aryusupova@aryusupova:~$ git config --global commit.gpgsign true
aryusupova@aryusupova:~$ git config --global gpg.program $(which gpg)
aryusupova@aryusupova:~$ gh auth login
? Where do you use GitHub? GitHub.com
? What is your preferred protocol for Git operations on this host? HTTPS
? Authenticate Git with your GitHub credentials? Yes
? How would you like to authenticate GitHub CLI? Login with a web browser

! First copy your one-time code: 0179-9FB6
Press Enter to open https://github.com/login/device in your browser...
/ Authentication complete.
- gh config set -h github.com git_protocol https
/ Configured git protocol
! Authentication credentials saved in plain text
/ Logged in as alyusupova
```

Рисунок 2.9: Связь репозитория с аккаунтом

Видим, что вход через терминал на git прошёл успешно

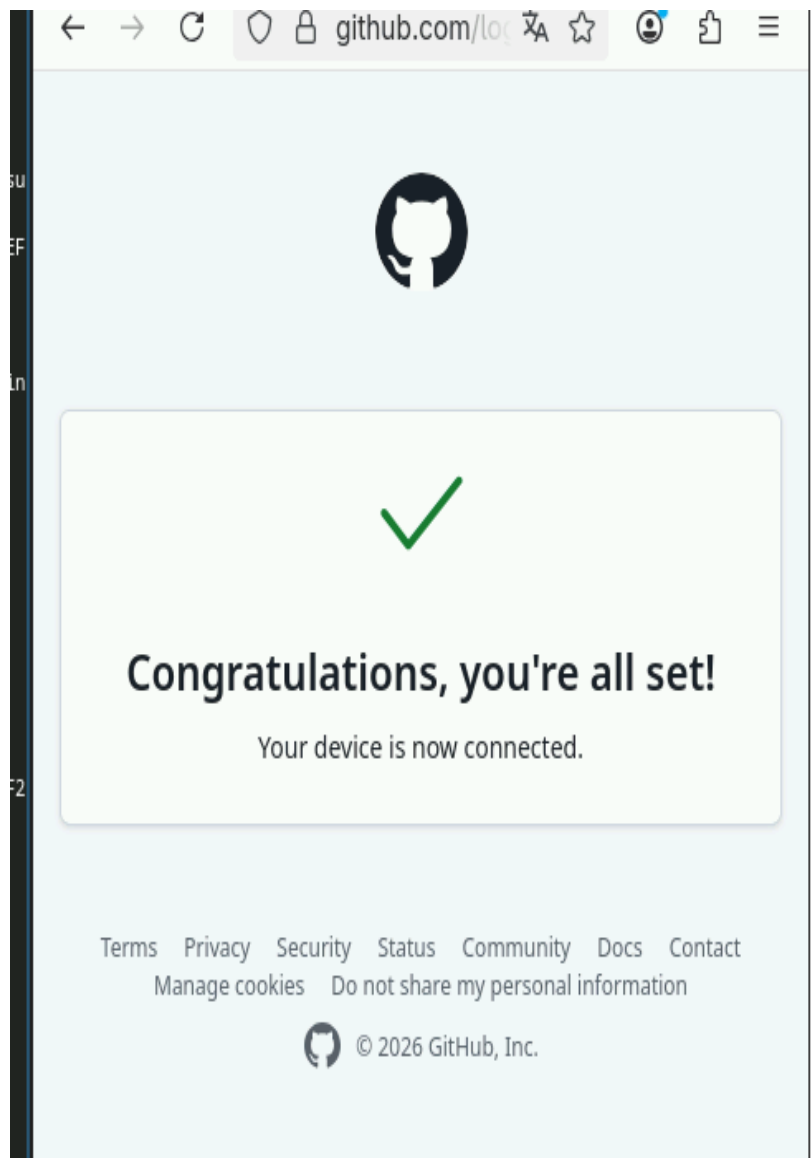


Рисунок 2.10: Успешный вход на git

Загрузка шаблона репозитория и синхронизация


```
✓ Logged in as alyusupova
alyusupova@alyusupova:~$ mkdir -p ~/work/study/2025-2026/"Операционные системы"
alyusupova@alyusupova:~$ [GFX1-]: RenderCompositorSWGL failed mapping default framebuffer, no dt
alyusupova@alyusupova:~$ cd ~/work/study/2025-2026/"Операционные системы"
alyusupova@alyusupova:~/work/study/2025-2026/Операционные системы$ gh repo create study_2025-2026_os-intro --template=yamadharma/course-directory-student-template --public
✓ Created repository alyusupova/study_2025-2026_os-intro on github.com
https://github.com/alyusupova/study_2025-2026_os-intro
alyusupova@alyusupova:~/work/study/2025-2026/Операционные системы$
```

Рисунок 2.11: Загрузка шаблона

Подготовка репозитория и коммит изменений

```
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы$ cd ~/work/
study/2025-2026/"Операционные системы"/study_2025-2026_os-intro
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-
2026_os-intro$ rm package.json
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-
2026_os-intro$ echo os-intro > COURSE
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-
2026_os-intro$ make
Usage:
  make <target>

Targets:
  list           List of courses
  prepare       Generate directories structure
  submodule      Update submules

aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-
2026_os-intro$
```

Рисунок 2.12: Первый коммит

```

2026_os-intro$ git add .
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$ git commit -am 'feat(main): make course structure'
Author identity unknown

*** Пожалуйста, скажите мне кто вы есть.

Запустите

git config --global user.email "you@example.com"
git config --global user.name "Ваше Имя"

для указания идентификационных данных аккаунта по умолчанию.
Пропустите параметр --global для указания данных только для этого репозитория
.

fatal: не удалось выполнить автоопределение адреса электронной почты (получен
о «aryusupova@aryusupova.(none)»)
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$ git config --global user.email '1032253510@pfur.ru'
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$ git config --global user.name 'alyusupova'
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$ git commit -m 'feat(main):make course structure'
[master f5698de] feat(main):make course structure
2 files changed, 1 insertion(+), 14 deletions(-)
delete mode 100644 package.json
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$ git push
Перечисление объектов: 5, готово.
Подсчет объектов: 100% (5/5), готово.
При сжатии изменений используется до 2 потоков
Сжатие объектов: 100% (2/2), готово.
Запись объектов: 100% (3/3), 946 bytes | 946.00 KiB/s, готово.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/alyusupova/study_2025-2026_os-intro.git
19ed786..f5698de master -> master
aryusupova@aryusupova:~/work/study/2025-2026/Операционные системы/study_2025-2026_os-intro$

```

Рисунок 2.13: Первый коммит

3 Вывод

Мы приобрели практические навыки работы с сервисом github.

4 Контрольные вопросы

1. Что такое системы контроля версий (VCS) и для решения каких задач они предназначаются?

Системы контроля версий (Version Control System, VCS) применяются при работе нескольких человек над одним проектом. Обычно основное дерево проекта хранится в локальном или удалённом репозитории, к которому настроен доступ для участников проекта. При внесении изменений в содержание проекта система контроля версий позволяет их фиксировать, совмещать изменения, произведённые разными участниками проекта, производить откат к любой более ранней версии проекта, если это требуется

2. Объясните следующие понятия VCS и их отношения: хранилище, commit, история, рабочая копия.

- хранилище - пространство на накопителе где расположен репозиторий
- commit - сохранение состояния хранилища
- история - список изменений хранилища (коммитов)
- рабочая копия - локальная копия сетевого репозитория, в которой работает программист. Текущее состояние файлов проекта, основанное на версии, загруженной из хранилища (обычно на последней)

3. Что представляют собой и чем отличаются централизованные и децентрализованные VCS? Приведите примеры VCS каждого вида.

Централизованные системы контроля версий представляют собой приложения типа клиент-сервер, когда репозиторий проекта существует в единственном экземпляре и хранится на сервере. Доступ к нему осуществлялся через специальное клиентское приложение. В качестве примеров таких программных продуктов можно привести CVS, Subversion.

Распределенные системы контроля версий (Distributed Version Control System, DVCS) позволяют хранить репозиторий (его копию) у каждого разработчика, работающего с данной системой. При этом можно выделить центральный репозиторий (условно), в который будут отправляться изменения из локальных и, с ним же эти локальные репозитории будут синхронизироваться. При работе с такой системой, пользователи периодически синхронизируют свои локальные репозитории с центральным и работают непосредственно со своей локальной копией. После внесения достаточного количества изменений в локальную копию они (изменения) отправляются на сервер. При этом сервер, чаще всего, выбирается условно, т.к. в большинстве DVCS нет такого понятия как «выделенный сервер с центральным репозиторием».

4. Опишите действия с VCS при единоличной работе с хранилищем.

Один пользователь работает над проектом и по мере необходимости делает коммиты, сохраняя определенные этапы.

5. Опишите порядок работы с общим хранилищем VCS.

Несколько пользователей работают каждый над своей частью проекта. При этом каждый должен работать в своей ветки. При завершении работы ветка пользователя сливается с основной веткой проекта.

6. Каковы основные задачи, решаемые инструментальным средством git?

- Ведение истории версий проекта: журнал (log), метки (tags), ветвления (branches).
- Работа с изменениями: выявление (diff), слияние (patch, merge).
- Обеспечение совместной работы: получение версии с сервера, загрузка обновлений на сервер.

7. Назовите и дайте краткую характеристику командам git.

- git config - установка параметров
- git status - полный список изменений файлов, ожидающих коммита
- git add . - сделать все измененные файлы готовыми для коммита.
- git commit -m «[descriptive message]» - записать изменения с заданным сообщением.
- git branch - список всех локальных веток в текущей директории.
- git checkout [branch-name] - переключиться на указанную ветку и обновить рабочую директорию.
- git merge [branch] — соединить изменения в текущей ветке с изменениями из заданной.
- git push - запустить текущую ветку в удаленную ветку.
- git pull - загрузить историю и изменения удаленной ветки и произвести слияние с текущей веткой.

8. Приведите примеры использования при работе с локальным и удалённым репозиториями.

- git remote add [имя] [url] — добавляет удалённый репозиторий с заданным именем;
- git remote remove [имя] — удаляет удалённый репозиторий с заданным именем;
- git remote rename [старое имя] [новое имя] — переименовывает удалённый репозиторий;

- `git remote set-url [имя] [url]` — присваивает репозиторию с именем новый адрес;
- `git remote show [имя]` — показывает информацию о репозитории.

9. Что такое и зачем могут быть нужны ветви (branches)?

Ветвление — это возможность работать над разными версиями проекта: вместо одного списка с упорядоченными коммитами история будет расходиться в определённых точках. Каждая ветвь содержит легковесный указатель HEAD на последний коммит, что позволяет без лишних затрат создать много веток. Ветка по умолчанию называется `master`, но лучше назвать её в соответствии с разрабатываемой в ней функциональностью.

10. Как и зачем можно игнорировать некоторые файлы при `commit`?

Зачастую нам не нужно, чтобы Git отслеживал все файлы в репозитории, потому что в их число могут входить: